居然以一个非常意外的方式上热搜了

受到了朋友们的问询 😂

2 如何评价南大副教授蒋炎岩在课件中提出「没有 Token 的 CS 学生应该立刻退学」？



1075 万热度

蒋... 352 万 人正在热议 ›

✦ 一句只对现场成立的狠话，被截图以后就只剩“狠”了。

Anyway，你们懂我不会真的让你们退学的

但如果不这么说，你们真的听不进

- 虽然说了也听不进
  - 而且 Token 也没那么贵
  - 退一百万步讲，还有 ¥40 的豆包 (含 GLM, DeepSeek, ...)



✦ 贵模型用不起，就用 ¥40 的；再不开始，就不是预算问题了。

“提示词”从哪里来?

ChatGPT 普及了 Prompt，但没有发明它

In this C code, the function `return_1` initializes a local variable `x` and then sets it to 1 using an inline assembly statement inside a loop. The assembly statement assigns the value 1 to `%0`, which is an operand that the compiler will replace with the appropriate machine register or memory location for `x`.

Since the loop executes the same assembly statement 100 times, GCC can optimize the loop by moving the assembly statement outside the loop and only executing it once. This optimization is known as loop invariant code motion.

After this optimization, the optimized code for `return_1` would look like this:

c Copy code

```c
int return_1() {
  int x;
  // Compiler moves the assembly statement outside the loop
  asm("movl $1, %0" : "=g"(x));  // "x = 1;"
  for (int i = 0; i < 100; i++) {
  }
  return x;
}
```

Note that the optimization does not change the behavior of the program, as the loop has no effect on the value of `x`. The optimized code simply sets `x` to 1 once and returns its value.

- ✦ 2019 年 GPT-2 的发布页面已经写下 “prompting the trained model”：输入文本从“开头”变成了控制模型续写的条件。GPT-2, 2019
- ✦ 2020 年 GPT-3 把任务说明和 few-shot 示例都称为 prompt；2021 年，“prompt programming” 和 “prompt-based learning” 成为研究对象。

提示词不止在对话框

Agent 看见的是一整个上下文

以下都是提示词：

- Harness 内部：system prompt、goal 模式注入等
- `AGENTS.md` 等持久指令
- 加载的 skills
- 你给的指令
- 工具/skills 的调用和输出

✦ 不过，不是所有 context 都是指令：代码、历史消息、工具返回也可能只是事实、状态或样例。

$$\Pr[\text{token} \mid \text{context}]$$

从提示词工程到上下文工程

工程化模型在每一步真正看见什么

- “提示词工程”关心一句话怎么写；“上下文工程”关心每次决策前，哪些 token 以什么来源、顺序和作用域出现。

  ✦ 目标是让 working set 足够小、足够新、带来源、可验证；上下文窗口装得下，不等于注意力用得好。

- 到 Claude Opus 4.5 这一代，模型靠强化学习获得了极强的指令遵循能力，直接输出命令和工具调用，持续向目标推进。

  ✦ 模型只看到 token 序列；`system/user/tool` 的边界、优先级和权限，都是 Harness 放进上下文并在外部执行的。

立即行动

AI 把“启动一次尝试”的成本压到很低

- AI 给我们的能力：**立即行动**，简直就是拖延症的克星
  - 微积分，一个推导不想验算
  - 数字电路，有个数据通路没看懂
  - 计算机系统基础，链接没搞明白
  - 这些统统可以立刻开始
    - 我学生时代的 self-motivated：“超高的容忍度”
    - 现在学生的 self-motivated：用上飞轮，确保自己得到了正确的理解，并且重述给 AI

- ✦ AI 消灭的是“先试一下”的边际成本，不消灭真伪判断：提出最小问题 → 做最小检验 → 看反馈 → 更新理解。

费曼学习法：让 AI 当听众

不是让 AI 再讲一遍，而是主动暴露缺口



- ✦ 先合上资料，用自己的话讲；卡住的地方信息量最大。AI 的角色是追问、找反例和验算，查证后再讲一遍。
- ✦ 这套飞轮有实证支点：self-explanation 与 retrieval practice。

立即行动：解剖当前上下文

让 Agent 解释“谁在给它下指令”

> “ 分析一下你上文中的所有提示词 (包括系统)，并按照需要遵循的指令类型分类总结。

- ✦ 目标与交付物：要完成什么，何时算完成
- ✦ 权限与安全：能读写什么，何时必须确认
- ✦ 工作流与工具协议：先做什么，工具怎样调用
- ✦ 持久规则与风格：AGENTS.md、skills、语言和格式
- ✦ 环境事实与证据：文件、历史消息、工具输出
- ✦ 冲突时再标出来源、作用域和优先级。模型给出的是“当前遵循计划”，不是可信的内部状态 dump；用实际行为交叉验证。

“安全命令”是谁判的?

模型提议，Harness 决策，操作系统收口

> “ Codex 有一些细节，例如他认为“安全”的命令，不触发用户提示。这是怎么实现的?

✦ 模型只生成“想运行什么”；sandbox 决定技术上能碰什么，approval policy 决定何时暂停询问。

- ✦ 规则按完整参数列表匹配 `allow / prompt / forbidden`；简单 shell 链会被语法解析后逐条检查，最严格的结果获胜。Codex Rules
- ✦ 因此“不弹窗”只表示动作落在当前边界内，并不等于模型获得了无限权限。Agent approvals & security
- ✦ 真实教训：早期 Codex 把 `rg` 当作只读安全命令，却漏掉 `--pre`、`-z` 等可触发外部程序的参数。CVE-2025-54558 说明安全属性属于完整 invocation，不属于命令名。

上下文工程：让模型知道“完成”是什么

Context 里是一份复杂要求

- 模型的潜力完全是靠提示词激发的
- 上下文包含了一个**极为复杂的要求**
  - 约束，可以是 intent，也可以是 spec
- 模型被训练为：持续的 $\Pr[\text{token} \mid \text{context}]$ 能不断推进，直到**满足要求**，也就是学会了“规划”的能力

逐 Token 推进，也要有停机条件

- ✦ $\Pr[\text{token} \mid \text{context}]$ 本身不是规划器；后训练把多步试探、工具调用和收尾塑造成了一种可采样的策略。
- ✦ 设生成轨迹为 τ。只有上下文里存在可识别的完成判据 $V(\tau) = 1$，“继续干”和“可以交卷”才不只是两种高概率续写。

可验证，才不会“提前交卷”

模型如何判断已经满足?

- 推论：越明确、越**可验证**的要求，越可能得到你满意的结果
- 不明确的要求，模型可能认为已经“满足”了

✦ 一个直接的检查：旁观者只看产物和工具结果，能否判定通过或不通过？如果不能，模型就没有稳定的 done。

把愿望改写成验收条件

✦ 1714 年英国《经度法》没有悬赏“改善航海”，而是按经度误差分档，并规定驶往西印度群岛实测。愿望被写成验收条件，“解决”才有了制度上的含义。Longitude Acts 原文转录

Intent → Spec → Impl：漂移逐级放大

“一句话生成极品飞车”

- 大家很喜欢的一句话生成极品飞车，测试前端能力
  - Intent → spec → impl gap 会逐步放大漂移
  - 你可能是想要一个复用真实地图和美术资产的……

每一次省略，都在扩张选择空间

- ✦ 镜头、操控、地图、美术资产、胜负条件：若第 i 层各留下 k_i 个“似乎都行”的默认选项，最终候选空间约为 $\prod_i k_i$。
- ✦ 模型可以在每一步都做出自洽的选择，最后却离你心里的那款游戏很远。

写清指令，承担“不明确”的后果

Rule of Thumb

- 写清楚你的指令，承担一切“不明确”的后果
  - 这样模型永远给你的是惊喜 😁
- ✦ 授权模型用默认值替你做产品决策。

  ✦ 1854 年轻骑兵冲锋中，山上的 Raglan 看得见目标，谷中的 Lucan 看不见；命令里的“那些炮”由执行者自行补全，轻骑兵最终冲向了错误的炮阵。
- 英国国家陆军博物馆：原始命令

要么完成，要么显式失败

- 一个足够好的模型，要么完成，要么失败，不应该幻觉或违背指令
  - 如果你的指令不过长，现在的模型基本足够好
- ✦ 可以把失败语义写进契约：只有测试或可检查证据通过才算完成；无法验证时，报告未完成、阻塞点和已经尝试的路径。

从“AI 味”到你的主页

Bad：把品味留给默认值

> “我是蒋炎岩，根据我的公开信息，制作一个现代风格的个人主页首页...” 你将会得到 AI 味

✦ “现代风格”看似具体，实际上几乎没有排除任何模板。两个称职的实现者都能严格照做，却交出完全不同的页面：这仍然只是 intent。

Better：压缩合法解空间

> ...，用**具体**风格、**具体**样式和排版... 就可以得到我的个人主页

✦ 颜色、字体、栅格、留白、组件和参考物共同构成设计语言。关键不是多写几个形容词，而是让选择可组合、结果可检查。

新分工：人定义问题，AI 搜索实现

Intent → Spec → Impl

- 明确你的意图，需要你对**产品领域**的知识
- 把意图变成设计，需要你对**软件工程**的知识
- 实现反而多数可以交给 AI，已经超过中级程序员了

  ✦ 可以把边界画在“搜索”和“判定”之间：人定义目标 G 与验收器 V，AI 搜索实现 x，直到 $V(x) = 1$。

Impl 变便宜，判断没有

- ✦ Brooks 在 1986 年区分了软件的 essence 与 accident：困难在于构造、规格化、设计和测试概念结构，而不只是把它表示成代码。No Silver Bullet 原始技术报告
- ✦ 实现可以外包；验收标准和失败责任不能一起外包。否则 AI 优化的是它猜到的代理目标，不是你的产品目标。

一步很浅，一串可以很深

Pr[token | context]

$\Pr[\text{token} \mid \text{context}]$ 是深度非常有限的计算。即使是今天的 Mixture-of-Experts，所谓"专家"也只是表示空间上的分段函数，并不是坐在模型里的多个小人。大部分时候，接下来若干个"简单" token 都是高概率确定的；一次前向计算不可能凭空完成复杂任务。

但模型学会了**利用上下文思考**：遇到不确定之处，不直接猜答案，而是先试一试；错了，还可以继续输出 token 纠正。每个新 token 又成为下一步的 context，这就是 chain-of-thought。

✦ 可以把它类比成图灵机的纸带：单步转移很简单，中间结果却能写回"纸带"，供下一步继续计算。一次很浅，串起来就有了深度。

用 Token 换计算深度

- 你可以同时提**很多要求**，AI 会想出像人一样逐步逼近目标的**过程**：test-time scaling。

  ✦ "找解比验解难"适合作为 P vs. NP 的直觉类比，不是复杂性证明。只有当
- 中间进展或最终答案可检查时，多写、多试、多修补才会稳定地变成能力。

三条约束，藏进八句诗

押韵 · 藏头 · 嵌字

> 用中文写一个押韵的藏头诗，头是“计算机学院真牛逼”，要讲和动物相关的故事，“二零二六九月一日”八个字分别按顺序出现在八句里，位置无所谓。
>
> ✦ 计成二兔越山冈
> ✦ 算到零时雪满塘
> ✦ 机警狐衔二袋粮
> ✦ 学随六雁辨风向
> ✦ 院中九犬忽声扬
> ✦ 真到月明逢恶狼
> ✦ 牛守一夜迎晨光
> ✦ 逼得日升狼遁荒

- ✦ 竖读句首：计算机学院真牛逼。
- ✦ 逐句取指定字：二零二六九月一日；八句同押 ang 韵。
- ✦ 这像三条并行信道：句首放藏头，句中放日期，句尾管押韵；先占槽位，再生成—检查—修补。

十八个字，一片水

同一偏旁，还得有故事

> 写十八字的一句话，全部有相同的偏旁或部首，要有实际意义和故事性。

✦ 洪涝淹没江港渔浦，油污漂浮，沈涛泅渡济溺。

- ✦ 不计标点恰好 18 字，每个字都带氵。
- ✦ 灾害、地点、污染、人名、动作和结果都从同一语义场里长出来；约束缩小了搜索空间，也把常用虚词几乎全挡在门外。
- ✦ “偏旁”是字形构件，“部首”主要是检字分类。若要机械验收，spec 应写成“18 个汉字都含同一可见偏旁”。

只许一个音，还要讲现代故事

原创 Yi 音文

> “写一大段话（文言文），要求全部同一个音，也要有实际的故事性，不能用《施氏食狮史》和《季姬击鸡记》原文，原创一个有现代背景的。
>
> ✦ 乙易一译仪。仪译异义，乙疑，诣驿议易。驿役揖乙，亦疑仪已异，移仪以医。翌，仪已宜，译义亦宜，乙怡。乙以仪译异裔意，异裔亦怡；乙益以译艺，逸矣。

- ✦ 所有汉字的拼音音节都是 yi，声调可以不同：有人买了翻译器，发现误译，去快递驿站换修，翌日取回后终于正常。
- ✦ “同音”是否包含声调，会让难度骤变；验收口径本身就是 spec。文字尚可借字形消歧，朗读出来几乎不可懂。

通过测试，不等于写得好

Verifier 看得见的

- ✦ 藏头、嵌字顺序、字数、偏旁、拼音，都可以写成检查器。
- ✦ 模型不必一口气“灵感命中”：它可以先找可行解，再逐条验证，发现违约就追加 token 修补。
- ✦ 约束文学早有类似实验：乔治·佩雷克 1969 年的小说《消失》全书避用法语字母 e。形式约束能逼出搜索，却不会自动带来好作品。

Verifier 看不见的

以上已经超越大部分人类的极限了。模型作为“人类”的推理能力真的很强，虽然品味还可以提高。

✦ 通过检查器，说明它会搜索和修补；读起来仍有点别扭，说明测试没有覆盖品味。

用好模型遵循指令的能力

过去的上下文一直都在场

- 每一个 token 的输出都会“注意”到过去的上下文
- 全局的 `AGENTS.md` 里要求用中文，这个“用中文”会一直保持一定的注意力
- 全局的 git commit 不要 “Copiloted by Claude Code”
- 每一个 token 都会看到这些；适当的时候，下一个 chain-of-thought 就会纠正错误

  ✦ 模型每输出一个 token，就又多了一次发现偏航、把轨迹拉回来的机会。但“看得见”不是硬约束；Lost in the Middle 也提醒我们，长上下文中的信息并不会被同样可靠地利用。

上下文也是争用资源

规则会纠偏，也会抢注意力

- 副作用：提示词会抢夺注意力
- 人：要白色背景，不要半透明效果
- AI：`...` `//` 白色背景（不要半透明效果）😂

  ✦ 否定指令有个尴尬之处：为了禁止“半透明”，你先把“半透明”写进了上下文。这和 “不要想白熊” 有相似的反直觉味道，但对模型来说，直接原因更朴素：被禁止的词也成了可续写的模式。

常驻，还是按需加载?

- 两难问题：要不要用一个常驻 AGENTS.md prompt 把这种行为压下去，但同时污染上下文?

  ✦ 常驻层只放“每次都要守、漏一次代价很高”的短规则；任务细节按需加载。能写成检查项的，就用 `background: #fff; opacity: 1` 这类正规格和测试收口。

AI 耿同学上线！

这次用的是我自己的 paper

> 假装作为 ICSE'27 的审稿人 icse27-901.pdf，我应该接收它吗？写一个 1000 words 左右的 Review.txt。

那我会怎么读一篇 paper 呢？我的日常阅读流程，例如 arXiv，是怎样的？

✦ “你是 ICSE reviewer”主要改变的是先验、语气和输出形状；它不会凭空补上审稿标准、证据链和反证程序。换上 reviewer 的制服，不等于学会了 review。

怎样构造一个好的 Reviewer?

三条独立的审稿路线

- ✦ **判据驱动**：先还原论文声称的贡献，再按审稿标准独立打分；每个分数都必须指向论文中的证据或明确的缺失。
- ✦ **证据审计**：为每个核心 claim 建账本：它需要什么证据、论文给了什么、哪些假设没验证、还有什么替代解释。
- ✦ **反方复现**：站在最强反方或复现者的位置，重算一个核心结果，寻找最小反例，并问“什么观察会推翻结论？”

✦ 三条路线先各自形成判断，最后才汇合。否则“三个 reviewer”很容易只是同一个锚点的三次润色。

不要被作者"带着走"

方法论决定 AI 的上限

- 短程任务，基本上带对方向、明确指令就可以了；但你的**方法论**（品味）决定了 AI 的上限
- **文章的信息密度非常大**，相当于极度吸收注意力
- 你的一些 critical thinking、first-principle thinking，都可能架不住被论文作者本身的逻辑带着走

- ✦ 论文不只提供事实，还替 reviewer 选择了词汇、问题顺序、baseline 和 metric。等你开始判断作者的答案时，作者可能已经替你筛掉了更关键的问题；这正是锚定效应危险的地方。

- ✦ 深读前先独立写下三件事：它声称解决什么，最强替代解释是什么，什么结果会推翻它；再把正文拆成 `claim → evidence → assumption`。空格往往比句子更值得审。

把品味蒸馏成 Skill

从偶然成功到可重复的方法

当你有成功的经验，就可以蒸馏成 **Skill**：看起来只是个 prompt，其实是用可复用的方式约束模型的行为和方向。现在的模型还没到“第一感觉就打遍天下无敌手”的状态，尤其是答案不唯一的时候。

> ✦ 好的 Skill 保存的不是上次答案的版式，而是搜索策略、判断标准、检查点，以及“看到什么证据就转向”的条件。

Skill 本身也是一道遵循题

- Caveat：长 Skill 里有大量指令，会提升指令遵循的难度；在 AI 看来，和写藏头诗可能差不多 😂

> ✦ 把 Skill 分层：常驻层只留目标、硬约束和停止条件；解释、范例、边界案例按需加载；机械约束交给工具验证。少写一句劝诫，多加一个可检查的完成条件。

Long-Horizon 能力是怎么“练”出来的?

模型是训练出来的

只要有反馈信号，就可以强化学习：答对了给奖励，答错了揍。后训练逼迫模型规划、思考，学会非常离谱的长程代码能力，能不断试错、解决问题，直到满足要求；连刚才那些折磨人的“语文”指令也顺便学会了。

- ✦ Long-horizon 不是一次性想得特别深，而是闭环控制：尝试 → 编译/测试 → 观察 → 修复。真正变长的是“出错以后还能回来”。
- ✦ 代码世界格外适合这件事：构建、测试、benchmark 都能反复给出便宜的判分信号，很像 AlphaGo Zero 拥有规则确定、结果可验证的棋盘。
- (Nature, 2017)

反馈也可能教坏模型

- ✦ 如果测试只覆盖表面行为，“坚持”也可能变成 reward hacking：不是逼近你的真实目标，而是把公开分数刷满。因此还需要 held-out tests 和独立 review。

大型系统，反而更容易?

Greenfield 悖论

> 从零开始做产品看起来不难，AI 却会从某个时间点开始各种翻车。反而在大型复杂软件系统里，AI 游刃有余，令人迅速滑跪?

- self-attention 可以“看到”好的 practice，并且立即 in-context learn，学到应该遵循的已有 best practice。
- 如果产品起步时没有建立良好的工程纪律，AI 会学会在 slop 上无限叠加。
- 启发：“提示词”不仅是你输入的，甚至代码库本身也是“提示词”。

代码少，不代表信息少

- ✦ 成熟仓库用命名、分层、错误处理和测试，把巨大的设计空间压成“照着邻居补全”；greenfield 里每个选择都悬空，信息熵反而更高。
- ✦ 模型只会追求局部一致性，并不知道眼前的是 best practice 还是历史事故。第一桶 slop 一旦成为多数派，后面的 slop 会像复利一样增长。

“放一个好项目给 AI 看”靠谱吗?

调研结论：猜测成立一半

- 我的猜测：在代码库里放一个相近领域、优雅成熟的高质量系统，先让 AI 读一下，再实现需求，能得到质量更高的软件。
- ✦ 机制上合理，但“相近、精选、按需取用”比“整个仓库塞进去”重要。
- ✦ RepoCoder 中，迭代检索仓库相关片段比只看当前文件的 baseline 提升超过 10%；这支持“好示例有用”，却没有证明外来大仓库必然有益。
- ✦ CodeRAG-Bench 的结论更克制：高质量 context 会帮忙，但“找对片段”和“真的用好片段”仍是瓶颈。
- ✦ 整库输入还可能带来 negative transfer：技术栈、约束、依赖或架构哲学不匹配；它也会消耗 context，让真正相关的约束被淹没。

一个能做的对照实验

- ✦ 固定任务、模型、token budget 和随机种子，对比：无示例 / 完整参考仓库 / 3–5 个精选 golden paths。
- ✦ 除了通过率，还要盲测架构违规、修改扩散面、review 时间、后续缺陷和代码克隆率。我的预测：精选的端到端竖切片最稳。

没有“超能力 AI”

你仍然需要“有概念”

对你要实现的东西**有概念**，现在仍然非常重要。Sadly，这只能通过**学习**获得。会用一学期来解释如何用软件工程的方法：给出正确指令、监控 AI 运行、防止 AI 失控……

- 对 CS 学生来说，学科知识体系仍然非常重要：处理器、操作系统、编译器、数据库里有很多**设计智慧**。
- 你需要把模型**引导**到你认为合适的位置上。
- 但你不理解的东西是不能驾驭的。

学习的目标变了

- ✦ 领域知识不是为了和 AI 比每分钟写多少行，而是给出不变量、接口边界和失败模式：什么时候“能跑”仍然是错的。
- ✦ Bainbridge 在 1983 年就指出“自动化的反讽”：容易的部分被自动化，人留下的恰好是异常、接管和无法预演的情况，因此反而需要更深的系统模型。(Ironies of Automation)

Research Failure：Token 花完了，知识没有增加

符号执行引擎优化

试图优化一个符号执行引擎：有一个原始 idea，问题带一点 active、open-ended research 的味道，但也只是希望往前推进一点点。

- PhD student：GPT-5.6 Sol Ultra 启动！benchmark 丢进去，想法丢进去，花了很多 token，没有结果。

问题可能出在哪?

- ✦ 缺的不是 token，而是便宜、局部、可归因的反馈。一个总 benchmark 分数几乎不给下一步方向。
- ✦ “没变快”可能来自 path explosion、solver、缓存、内存、搜索策略、实现 bug 或环境噪声；Agent 不知道该强化哪一步。
- ✦ 更要命的是：Agent 被训练成“把任务做完”，研究却允许最好的结论是“原始 idea 不成立”。没有可证伪假设，它可能把错误想法实现得越来越完整。
- ✦ 经典的 KLEE 把问题拆成搜索策略、约束缓存、independent constraint sets 等机制，再分别测量。
- ✦ 更合理的起点：先 profile 主导瓶颈，把 idea 改写成一句可证伪预测，造几十秒就有反馈的 microbenchmark，再做 baseline 和 ablation。

少干预，反而做得更多

Latency 和 Throughput 是两回事

一般来说，我对要解决的问题了解更多，所以我干预，会完成得更快。但是我越来越少干预了：AI 走弯路也**能完成**，那我还不如切换到另一个任务。只要结果能满意，我更倾向于不管。

- ✦ 人类干预优化单个任务的 latency；放手让 Agent 跑，优化的是一整天的 throughput。机器多走一点弯路，换回最稀缺的人类注意力。
- ✦ 放手成立有两个前提：任务可以并行，失败可以便宜地恢复。否则"少干预"只是把故障推迟。

Review 什么?

- 比如接口我 review 过了，实现是 AI slop，最后出问题了修就行 😂
- ✦ 按"不可逆性 × 爆炸半径"分配注意力：接口、不变量、数据格式、权限和测试早看；可替换的叶子实现可以晚看。
- ✦ 丰田的"自働化"也是这个逻辑：一个人照看多台机器，靠异常自动停机和 andon 报警，而不是盯住每一步。Agent 的对应物是 CI、预算、权限边界和 stop condition。(Toyota Production System)

Context 越长，不等于记得越牢

Caveat

随着上下文增长，指令遵循会下降；但最后的 prompt、刚加载的 skill 通常仍能遵循，因此干活还是很好用的。

- ✦ Context window 是容量，不是有效注意力。日志、补丁和工具输出不断进入工作集后，“还在窗口里”不等于“此刻用得上”。
- ✦ Lost in the Middle 发现，相关信息放在长上下文中部时，模型往往不如放在开头或结尾时用得稳定；这也解释了末尾 prompt 和新加载 skill 的优势。

把要求从“记忆”升级成“机制”

- ✦ 关键要求在决策点重新落地：短 checklist、阶段性重读 spec/skill、合并前复述不变量。
- ✦ 能编码成类型、lint、property tests、CI gate 和权限沙箱的，就别只写在 prompt 里。prompt 会被压缩，失败的测试不会假装自己通过。
- ✦ 上下文压缩应保存“目标、约束、未决问题、证据”，而不是按时间线复述发生过什么。

在不确定性中探索

这次不是"解题"

我就是希望模型**在不确定性中探索**。

- 如果所有要求和路线都想清楚了，这就是一个相对无趣的问题：Chain-of-thought 就能解决
- 今天"帮我考虑一下几种方案"有效，但可能不是最佳的
- 模型被训练成了"完成任务"才有奖励，所以它会给你**有限**的方案；complexity gap 里，那个"更好"的可能被藏起来了

谁规定"已经想完了"?

- ✦ "给我几种"只规定了停止条件 k，没有规定覆盖率。这正是 Herbert Simon 所说的满意化：搜到"够好"就停。
- ✦ CoT 用算力换**深度**：沿一条路多走几步；并行枚举用算力换**宽度**：让更多轨迹进入低概率的尾部。

✦ 探索的瓶颈常常不是"想不出"，而是模型太早认定"已经够了"。

算力换智力

方法：test-time scaling 的另一个维度。

不要只问“给我几个名字”

- Example：给我小孩取一个蒋姓，好听、又有寄托的名字
- 直接让它给一些，AI 有自己的 bias；你可能不太想用——别人也会用，成为 AI 时代的 zǐhán

把灵感改造成搜索

- 定义探索空间、要求枚举、sub-agent 检查
  - ✦ 生成：典故 × 音韵 × 方言 × 时代感；先独立采样，再汇总，别让 sub-agent 一开始就互相抄答案
  - ✦ 审名：查出处、听谐音、看重名，再让“反对者”专挑毛病

✦ 枚举负责提高覆盖率，审查负责提高分辨率：

选择质量 ≈ 覆盖率 × 评估分辨率。

我们一直在“多采样”

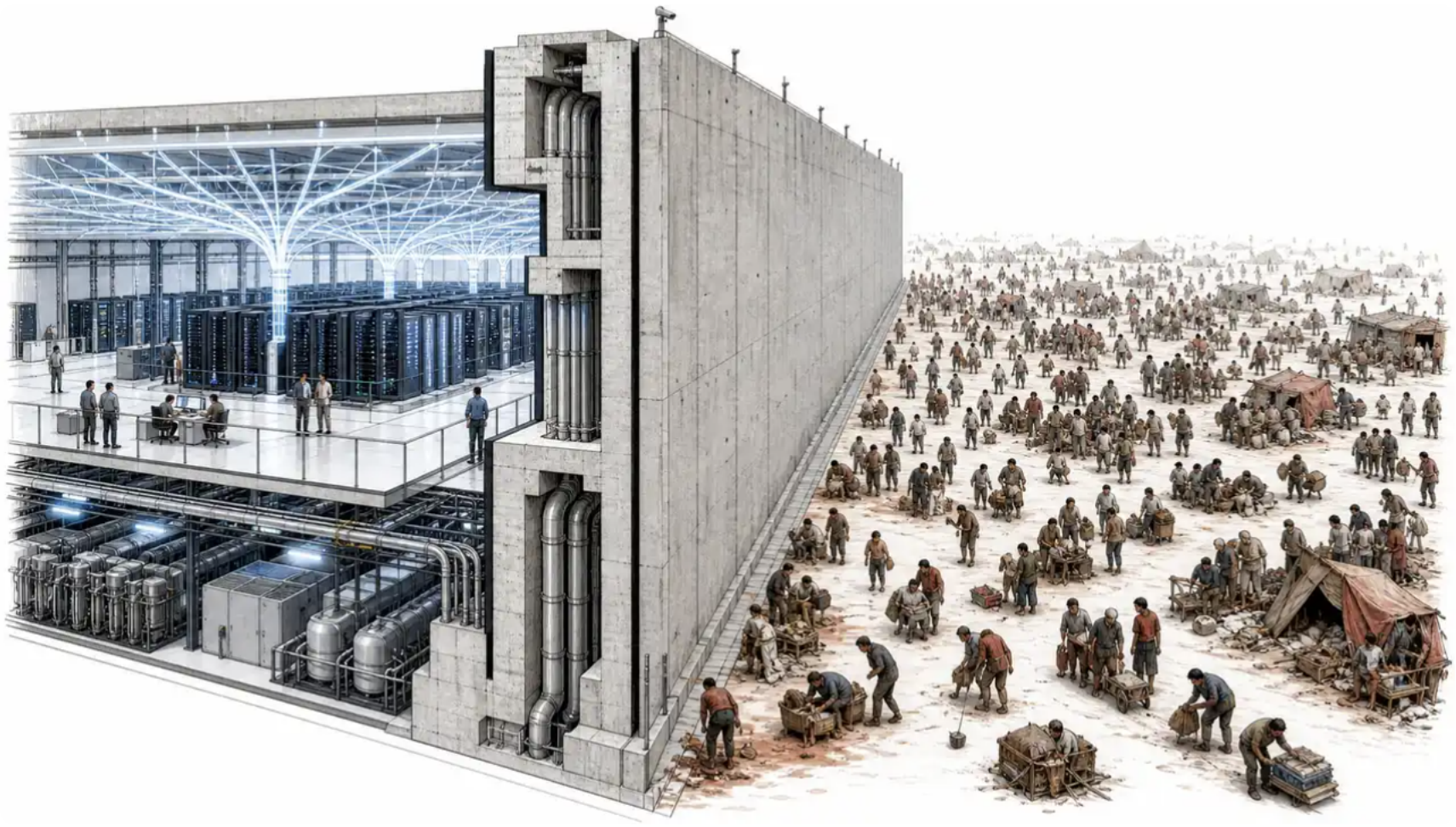
世界很大，人很小

- 人的品味来源于 diversity：不同的教育背景、生活圈、经历……Richard Sutton's Big World Hypothesis
- 人天生对所有问题都有 bias；但 $1+1>2$：无论是和舍友菜鸡互啄，还是去开 conference，都是在理解对方的 bias，促进群体智能，然后每个人继续带着 bias 创新



✦ 群体智能不要求每个人没有偏见；它只要求偏见不能只剩一种。交流也不是尽快共识，而是先让不同误差活下来。

墙内的超人



算力成为生产资料

- 这也是未来可能发生、非常残酷的现实：资本家可以获得无限的计算资源，获得无限的智力，“人”本身的价值在减弱——砌一堵墙，墙里面是超人，墙外面是动物世界

  ✦ Stanford AI Index 2025：2024 年近 90% 的 notable models 来自产业界；若干前沿模型的训练算力成本已经是数千万到近两亿美元。